

**Department of Computer Science and Software Engineering
The University of Western Australia**

CITS1401
Computational Thinking with Python
Project 1 Semester 1 2023
(Individual project)

Submission deadline: 6:00 PM 21 April 2023.

Project description:

You should construct a Python 3 program containing your solution to the following problem and submit your program electronically on Moodle. The name of the file containing your code should be your student ID e.g. 12345678.py. No other method of submission is allowed. Please note that this is an individual project. Your program will be automatically run on Moodle for sample test cases provided in the project sheet if you click the “check” link. However, your submission will be tested thoroughly for grading purposes after the due date. Remember you need to submit the program as a single file and copy-paste the same program in the provided text box. You have only one attempt to submit so don’t submit if you are not satisfied with your attempt. All open submissions at the time of the deadline will be automatically submitted. There is no way in the system to open the closed submission and reverse your submission.

You are expected to have read and understood the University's [guidelines on academic conduct](#). In accordance with this policy, you may discuss with other students the general principles required to understand this project, but the work you submit must be the result of your own effort. Plagiarism detection, and other systems for detecting potential malpractice, will therefore be used. Besides, if what you submit is not your own work then you will have learnt little and will therefore, likely, fail the final exam.

You must submit your project before the deadline listed above. Following UWA policy, a late penalty of 5% will be deducted for each day (or part day), after the deadline, that the assignment is submitted. No submissions will be allowed after 7 days following the deadline except approved special consideration cases.

Project Overview:

The XYZ organisation collected world population data for all countries over the world using different parameters such as population in 2020, annual change, net change, per kilometre density, land area, median age, etc. The data also shows the regions for each country as shown in Figure 1. It can be observed that each region has many countries and has its own population.

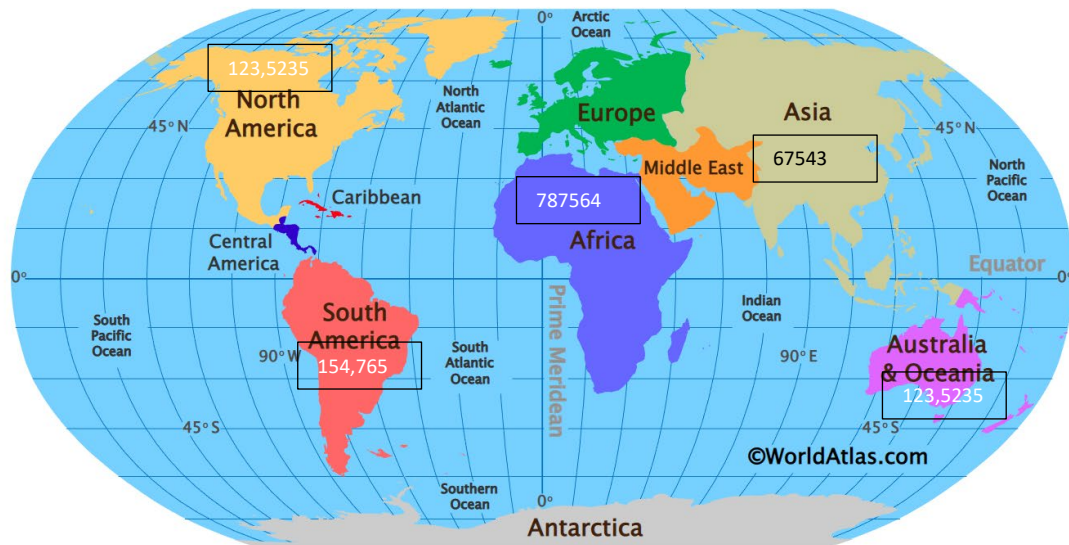


Figure 1: The world population of different regions

You are required to write a Python 3 program that will read a CSV file. After reading the file, your program is required to complete the following statistical tasks:

- 1) Find the country name which has minimum and maximum population in a specific region which has positive net change in population.
- 2) Calculate the average and standard deviation of population for a specific region.
- 3) Calculate the density of population for each country in a specific region.
- 4) Calculate the correlation between population and land area for all the countries in a specific region.

Requirements:

- 1) **You are not allowed to import any external or internal module in python.** While use of many of these modules, e.g. csv or math is a perfectly sensible thing to do in production setting, it takes away much of the point of different aspects of the project, which is about getting practice opening text files, processing text file data, and use of basic Python structures, in this case lists and loops.
- 2) Ensure your program does NOT call the `input()` function at any time. Calling the `input()` function will cause your program to hang, waiting for input that automated testing system will not provide (in fact, what will happen is that if the marking program detects the call(s), it will not test your code at all which may result in zero grade).
- 3) Your program should also not call `print()` function at any time except for the case of graceful termination (if needed). If your program has encountered an error state and is exiting gracefully then your program needs to return zero as the value for the correlation otherwise empty list and print an appropriate message. At no point should you print the program's outputs instead of (or in addition to) returning them or provide a printout of the program's progress in calculating such outputs.

- 4) Do not assume that the input file names will end in .csv. File name suffixes such as .csv and .txt are not mandatory in systems other than Microsoft Windows. Do not enforce that within your program that the file must end with a .csv or any other extension (or try to add an extension onto the provided csv file argument), doing so can easily lead to losing marks.

Input:

Your program must define the function `main` with the following syntax:

```
def main(csvfile, region):
```

The input arguments for this function are:

- `csvfile`: The name of the CSV file (as string) containing the record of the population of all countries in the world. Below is the first row of the sample file:

Name	Population(2020)	Yearly Change	Net Change	Land Area	Regions
Armenia	2,963,243	0.19%	5,512	28,470	Asia

- `region`: The dataset has multiple regions and their population details. It will be a string parameter containing name of a region.

Output:

Four outputs are expected in the order below.

- i) A list containing the name of countries having maximum and minimum population in the input region where net change in population is positive. Your output should be stored in a list in the following order:

[country with maximum population, country with minimum population]

For example: [India, Srilanka]

- ii) A list containing the average and standard deviation of population in the input region when distributed over the number of countries in the input region. Your output should be stored in a list in the following order:

[average, standard deviation]

For example: [300000, 15.6]

- iii) A list of a list containing the name and density of each country for the input region in descending order. Your output should be stored in a list of lists in the following order:

[[country1,density1] , [country2,density2] , ... [countryN, densityN]]

For example: [[china, 156], [India, 109], [Bangladesh, 73]]

- iv) A numeric value containing the correlation between the population and the land area for all the countries in the input region. The expected output is a single float value.

All returned numeric outputs (both in lists and individual) must contain values rounded to four decimal places (if required to be rounded off). Do not round the values during calculations. Instead round them only at the time when you save them into the final output variables.

Examples:

Download `countries.csv` file from the folder of Project 1 on LMS or Moodle. An example of how you can call your program from the Python shell (and examine the results it returns) is provided below:

```
>>> MaxMin, stdvAverage, density, corr = main('countries.csv',  
'Africa')
```

The output variables returned are:

```
>>> MaxMin  
['Western Sahara', 'Seychelles']  
  
>>> stdvAverage  
[291620.6667, 239936.4497]  
  
>>> density  
[['Mayotte', 727.5067], ['Sao Tome & Principe', 228.2906],  
['Cabo Verde', 137.9620], ['Saint Helena', 15.5821],  
['Seychelles', 11.0008], ['Western Sahara', 2.2456]]  
  
>>> corr  
0.6243
```

Assumptions:

Your program can assume the following:

- Anything that is meant to be string (e.g. header) will be a string, and anything that is meant to be numeric will be numeric.
- The order of columns in each row will follow the order of the headings provided in the first row. However rows can be in random order except the first row which contains the headings.
- No data will be missing in the CSV file; however values can be zero and must be accounted for when calculating averages and standard deviations.
- The `main()` function will always be provided with valid input parameters.
- The necessary formulas are provided at the end of this document.

Important grading instruction:

Note that you have not been asked to write specific functions. The task has been left to you. However, it is essential that your program defines the top-level function `main(csvfile, region)` (commonly referred to as “`main()`” in the project documents to save space when writing it. Note that when `main()` is written it still implies that it is defined with its two input arguments). The idea is that within `main()`, the program calls the other functions. (Of course, these functions may then call further functions.) This is important because when your code is tested on Moodle, the testing program will call your `main()` function. So if you fail to define `main()`, the testing program will not be able to test your code and your submission will be graded zero. Don’t forget the submission guidelines provided at the start of this document.

Marking rubric:

Your program will be marked out of 30 (10% of the total marks).

22 out of 30 marks will be awarded automatically based on how well your program completes a number of tests, reflecting normal use of the program, and also how the program handles various states including, but not limited to, different numbers of rows in the input file and / or any error states. You need to think creatively what your program may face. Your submission will be graded by data files other than the provided data file. Therefore, you need to be creative to look into corner or worst cases. I have provided few guidelines from ACS Accreditation manual at the end of the project sheet which will help you to understand the expectations.

8 out of 30 marks will be awarded on style (5/8) “the code is clear to read” and efficiency (3/8) “your program is well constructed and run efficiently”. For style, think about use of comments, sensible variable names, your name at the top of the program, student ID, etc. (Please watch the lectures where this discussed)

Style Rubric:

0	Gibberish, impossible to understand
1-2	Style is really poor or fair
3-4	Style is good or very good, with small lapses
5	Excellent style, really easy to read and follow

Your program will be traversing text files of various sizes (possibly including large csv files) so you need to minimise the number of times your program looks at the same data items.

Efficiency rubric:

0	Code too complicated to judge efficiency or wrong problem tackled
1	Very poor efficiency, additional loops, inappropriate use of readline()
2	Acceptable or good efficiency with some lapses
3	Excellent efficiency, should have no problem on large files, etc.

Automated testing is being used so that all submitted programs are being tested the same way. Sometimes it happens that there is one mistake in the program that means that no tests are passed. If the marker is able to spot the cause and fix it readily, then they are allowed to do that and your - now fixed - program will score whatever it scores from the tests, minus 4 marks, because other students will not have had the benefit of marker intervention. Still, that's way better than getting zero. On the other hand, if the bug is hard to fix, the marker needs to move on to other submissions.

Extract from Australian Computing Society Accreditation manual 2019:

As per Seoul Accord section D, a complex computing problem will normally have some or all of the following criteria:

- involves wide-ranging or conflicting technical, computing, and other issues;
- has no obvious solution, and requires conceptual thinking and innovative analysis to formulate suitable abstract models;

- a solution requires the use of in-depth computing or domain knowledge and an analytical approach that is based on well-founded principles;
- involves infrequently-encountered issues;
- is outside problems encompassed by standards and standard practice for professional computing;
- involves diverse groups of stakeholders with widely varying needs;
- has significant consequences in a range of contexts;
- is a high-level problem possibly including many component parts or sub-problems;
- identification of a requirement or the cause of a problem is ill defined or unknown.

Necessary formulas:

i) Correlation coefficient:

Mathematical formula to calculate correlation is as follows:

$$r = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2 \sum (y_i - \bar{y})^2}}$$

where x_i and y_i are the values of population and land area of each country respectively. $\bar{x}$ is the mean of population and $\bar{y}$ is the mean of land area.

ii) Standard deviation:

Mathematically, standard deviation is represented as:

$$s = \sqrt{\frac{\sum_{i=1}^N (x_i - \bar{x})^2}{N - 1}}$$

where $x_1, x_2, x_3 \dots x_n$ are observed value in sample data. $\bar{x}$ is the mean value of observations and N is the number of sample observations.

iii) Density:

Density of a country can be calculated using the following formula:

$$D = P/L$$

where P is the population of a country and L is the land area of a country.